

Parte 5

View / Thymeleaf no Spring Boot MVC

Como transformar dados do Model em páginas HTML dinâmicas, com formulário, listagem, validação e CSS.

Arquivos desta parte

lista.html

Renderiza a tabela de jogos, o botão de novo cadastro, a mensagem de sucesso e o estado vazio da tela.

formulario.html

Usa th:object, th:field e ação dinâmica para salvar ou editar o mesmo objeto Game.

style.css

Separa a apresentação visual da lógica do template e melhora a experiência de uso.

Objetivo didático

Mostrar não só o que a View faz, mas a sequência de construção até a página final.

Qual é o papel da View?

Ela recebe dados preparados pelo Controller e transforma isso em HTML para o navegador.

Exibir dados

Exemplo: a coleção games vira linhas da tabela em lista.html.

Receber entrada

O formulário captura nome, plataforma, gênero e status.

Renderizar feedback

Mensagens de erro e sucesso aparecem na própria interface.

Apoiar navegação

Links e ações apontam para as rotas do Controller.

Moral da história: a View não consulta banco nem decide regra de negócio. Ela apresenta.

Thymeleaf no centro da renderização

th:text substitui o conteúdo do elemento por um valor do Model

th:each repete um bloco para cada item da coleção

th:object define o objeto-base usado no formulário

th:field faz o binding entre campo HTML e atributo do objeto

th:if controla a renderização condicional

th:href / th:action gera URLs alinhadas às rotas da aplicação

Evolução da implementação da View

Ensine na ordem em que o aluno percebe valor: primeiro enxerga a tela, depois entende o motor por trás.

1 HTML base

Criar a estrutura HTML com título, card e áreas da página.

2 Lista dinâmica

Trocar conteúdo fixo por `th:each`, `th:text` e links com `th:href`.

3 Formulário

Ligar campos ao objeto Game com `th:object` e `th:field`.

4 Validação

Exibir erros de campo e mensagens de sucesso na interface.

5 CSS e polimento

Aplicar `style.css` para entregar uma aplicação com cara de projeto real.

Estratégia de aula: mostre a página pronta rapidamente, volte uma camada e reconstrua a tela em blocos. O aluno entende melhor quando liga cada atributo do Thymeleaf ao efeito visual que ele produz.

Estrutura em src/main/resources

resources

```
1 src/main/resources
2 |   static
3 |     |   css
4 |     |     style.css
5 |   templates
6 |     |   formulario.html
7 |     |   lista.html
```

templates

Arquivos HTML processados pelo Thymeleaf. O retorno do Controller aponta para esses nomes.

static

Arquivos estáticos servidos diretamente pelo Spring Boot, como CSS, JS e imagens.

Por que separar?

Template cuida da estrutura dinâmica. CSS cuida da aparência. Misturar tudo vira gambiarra gourmet.

carregamento do CSS

```
1 <link rel="stylesheet"
  th:href="@{/css/style.css}">
```

retornos do Controller

```
1 return "lista";
2 return "formulario";
```

Leitura didática

- quando o método retorna "lista", o Spring procura templates/lista.html
- o CSS em static/css fica acessível pela URL /css/style.css
- essa organização aproxima a prática de um projeto real

lista.html: exibindo a coleção de games

lista.html

```
1 <a th:href="@{/games/novo}" class="botao-principal">Novo jogo</a>
2
3 <div th:if="${mensagem}" class="mensagem-sucesso"
4     th:text="${mensagem}"></div>
5
6 <tr th:each="game : ${games}">
7     <td th:text="${game.nome}"></td>
8     <td th:text="${game.plataforma}"></td>
9     <td th:text="${game.genero}"></td>
10    <td><span class="badge" th:text="${game.status}"></span></td>
11    <td class="coluna-acoes">
12        <a
13            th:href="@{/games/{id}/editar(id=${game.id})}">Editar</a>
14        <form th:action="@{/games/{id}/excluir(id=${game.id})}"
15            method="post">
16            <button type="submit"
17                class="link-botao">Excluir</button>
18            </form>
19        </td>
20    </tr>
21
22 <tr th:if="${#lists.isEmpty(games)}">
23     <td colspan="5" class="linha-vazia">Nenhum jogo cadastrado
24     ainda.</td>
25 </tr>
```

th:each

Percorre a coleção enviada pelo Controller em `model.addAttribute("games", ...)`.

th:text

Preenche as células da tabela com os atributos de cada objeto Game.

th:href / th:action

Gera URLs de editar e excluir usando o id do registro atual.

Estado vazio e flash message

A View já trata lista vazia e feedback de sucesso, sem colocar HTML no Controller.

formulario.html: binding com o objeto Game

formulario.html

```
1 <h2 th:text="${modoEdicao} ? 'Editar jogo' : 'Novo
jogo' "></h2>
2
3 <form th:action="${modoEdicao} ?
@{/games/{id}/editar(id=${game.id})} : @{/games}"
4     th:object="${game}"
5     method="post">
6
7     <input type="hidden" th:field="*{id}">
8
9     <input id="nome" type="text" th:field="*{nome}">
10    <input id="plataforma" type="text"
th:field="*{plataforma}">
11    <input id="genero" type="text" th:field="*{genero}">
12
13    <select id="status" th:field="*{status}">
14        <option value="">Selecione</option>
15        <option value="Quero jogar">Quero jogar</option>
16        <option value="Jogando">Jogando</option>
17        <option value="Zerado">Zerado</option>
18    </select>
19 </form>
```

th:object="\${game}"

Define que o formulário inteiro trabalha com um objeto Game.

th:field="*{...}"

Liga cada campo ao atributo correspondente do objeto. Menos código manual, menos chance de erro.

Ação dinâmica

O mesmo formulário atende cadastro e edição. Quem decide é a flag modoEdicao.

Input hidden id

Na edição, o id acompanha o objeto e mantém o vínculo com o registro correto.

Validação e feedback visual na tela

erros de campo no formulário

```
1 <small class="erro"
2
th:if="${#fields.hasErrors('nome')}}"
3     th:errors="*{nome}"></small>
4
5 <small class="erro"
6
th:if="${#fields.hasErrors('plataforma')}}"
7
th:errors="*{plataforma}"></small>
8
9 <small class="erro"
10
th:if="${#fields.hasErrors('status')}}"
11     th:errors="*{status}"></small>
```

mensagem de sucesso na listagem

```
1 <div th:if="${mensagem}"
2     class="mensagem-sucesso"
3     th:text="${mensagem}"></div>
```

Ligação com o Controller

Quando BindingResult tem erro, o método retorna formulario novamente. O Thymeleaf reaproveita o objeto e mostra os avisos.

#fields.hasErrors

Verifica se o campo específico falhou na validação.

th:errors

Escreve a mensagem definida nas anotações da entidade Game.

Flash attribute

Depois de salvar, atualizar ou excluir, a tela de lista pode exibir uma confirmação amigável.

style.css: separando estrutura de apresentação

style.css

```
1 body {
2     background: #f3f5fb;
3     color: #1f2937;
4 }
5
6 .card {
7     background: #ffffff;
8     border-radius: 14px;
9     box-shadow: 0 10px 25px rgba(15, 23, 42,
10 0.08);
11     padding: 24px;
12 }
13 button, .botao-principal {
14     background: #2563eb;
15     color: #ffffff;
16 }
17
18 .badge {
19     background: #dbeafe;
20     color: #1d4ed8;
21 }
22
23 @media (max-width: 768px) {
24     table { min-width: 720px; }
25 }
```

Consistência visual

container, card, botões e badge mantêm a mesma identidade em todas as páginas.

Melhor experiência

Cores, espaçamento e sombra fazem o projeto parecer produto de verdade, não dever de casa triste.

Responsividade básica

O media query ajuda a página a degradar melhor em telas menores.

Mensagem importante

Thymeleaf cuida da renderização dinâmica. CSS cuida da aparência. Cada um no seu quadrado e a aplicação agradece.

Fluxo da View e roteiro de implementação em sala

Fluxo completo da renderização

- 1. Controller** retorna "lista" ou "formulario" e envia atributos ao Model
- 2. Thymeleaf** lê os atributos e processa th:text, th:each, th:field e afins
- 3. HTML final** o navegador recebe a página já montada
- 4. CSS** o arquivo style.css dá acabamento visual à resposta
- 5. Usuário** interage novamente e inicia outro ciclo de requisição

Roteiro curto para ensinar ao vivo

1 Montar lista.html

Criar cabeçalho, card e tabela. Depois substituir conteúdo fixo por th:each e th:text.

2 Montar formulario.html

Criar o form, ligar com th:object e mapear os campos com th:field.

3 Adicionar erros e feedback

Mostrar th:errors no form e mensagem de sucesso na listagem.

4 Aplicar CSS

Apresentar style.css como camada separada e explicar o ganho de manutenção.

Checklist da explicação

- de onde vem o atributo games
- como o objeto game entra no formulário
- por que o mesmo HTML serve para cadastro e edição
- onde o CSS é carregado